ÖVNINGSUPPGIFTER: LEKTION 3

Datamodellering, Många-till-många-relationer, Tabell-Alias och Normalisering

🟢 Gröna uppgifter (Relationsmodellering & Kopplingstabeller)

Uppgift 1

Incidentmappning (1:M) – Skriv ett DDL-skript som skapar tabellen `Incidents` i SQL Server. Tabellen ska ha en `PRIMARY KEY` (`Id` med `IDENTITY`), `Severity`, `Description` samt en `FOREIGN KEY` (`TargetServerId`) som länkar till `Servers.Id`.

Uppgift 2

Mata in Incidenter (DML) – Skriv SQL-frågor som matar in tre fiktiva incidenter för servern med `Id = 1`. En incident ska ha severity 'Critical', en 'High' och en 'Low'.

Uppgift 3

Hämta med precision (INNER JOIN) – Skriv en `SELECT`-fråga som hämtar alla incidenter och visar kolumnerna `ServerName`, `IpAddress`, `Severity` och `Description`. Använd en `INNER JOIN`.

Uppgift 4

Säkerhetsprofiler (M:M) – Skriv ett DDL-skript som skapar tabellen `SecurityProfiles` med kolumnerna `Id (PK, Identity)`, `ProfileName` och `EncryptionStandard`.

Uppgift 5

Kopplingstabellens grund (Composite PK) – Skriv ett DDL-skript för kopplingstabellen `ServerSecurityMapping` som binder samman `Servers` och `SecurityProfiles`. Sätt en sammansatt primärnyckel (`Composite PRIMARY KEY`) på kolumnerna `ServerId` och `ProfileId`.

🟡 Gula uppgifter (Normalisering & Anomalidetektering)

Uppgift 6

Anomalijägaren – Identifiera vilken typ av databasanomali (Insättnings-, Uppdaterings- eller Borttagningsanomali) som uppstår om du tvingas lagra en teknikers telefonnummer direkt på varje serverrad, och numret plötsligt ändras. Motivera ditt svar.

Uppgift 7

Krossa UNF till 1NF – Du har fått följande sträng i en cell: `Server: Prod-01 | IpAddresses: '10.0.0.15, 192.168.1.10'`. Skriv den SQL-kod (DML) som krävs för att lagra denna data enligt reglerna för Första Normalformen (1NF).

Uppgift 8

Från 2NF till 3NF – Skapa en ny tabell `DataCenters` med `Id (PK, Identity)`, `DataCenterName` och `DataCenterCity`. Skriv sedan en `ALTER TABLE`-sats som lägger till en `FOREIGN KEY` i `Servers`-tabellen för att lyfta arkitekturen till 3NF genom att eliminera transitiva beroenden.

Uppgift 9

Det säkra pipeline-schemat (SQLite) – Skriv ett DDL-skript anpassat för SQLite som skapar en lokal spegling av kopplingstabellen `ServerSecurityMapping`. Kom ihåg att justera datatyperna för SQLite-motorn.

Uppgift 10

Audit-loggning i pipelines – Skriv en `INSERT INTO`-fråga för SQLite som sparar en rad i en lokal `PipelineAuditLogs`-tabell och sätter tidsstämpeln automatiskt med den SQLite-specifika tidsfunktionen för nuvarande tidpunkt.

🔴 Röda uppgifter (Constraints, Cascading & Dataintegritet)

Uppgift 11

Affärslogik i kärnan (CHECK Constraint) – Skriv en `ALTER TABLE`-sats i SQL Server som lägger till en `CHECK`-constraint på tabellen `Incidents` så att kolumnen `Severity` *endast* tillåter värdena 'Low', 'Medium', 'High' eller 'Critical'.

Uppgift 12

Miljö-brandväggen – Skapa en `CHECK`-constraint på tabellen `Servers` för kolumnen `Environment` som blockerar alla strängar förutom 'Production', 'Staging' och 'Development'.

Uppgift 13

Hantering av raderingskedjor (SET NULL) – Konfigurera en Foreign Key mellan `Servers` och `Users` med regeln `ON DELETE SET NULL`. Skriv SQL-koden som demonstrerar vad som händer med servrarna om en användare raderas.

Uppgift 14

Den låsta dörren (Principle of Least Privilege) – Skriv ett skript som skapar en databasanvändare `Auditor_Role` utan login. Ge användaren exklusiv `GRANT SELECT`-behörighet på enbart tabellerna `Incidents` och `Servers`.

Uppgift 15

Säkerhetsisolering (1:1-Härdning) – *Extra utmaning:* Skapa en separat tabell `ServerSecretKeys` som har en 1:1-relation med `Servers`. Primärnyckel (`ServerId`) ska agera både PK och FK, och kolumnen `EncryptedKey` måste vara `UNIQUE`.